Interfaces adaptatives en SwiftUI

420-249-AH - Programmation mobile iOS

1. Qu'est-ce qu'une interface adaptative ?

Une interface adaptative est une interface capable de bien s'afficher sur différents appareils et différentes tailles d'écran.

Exemples :

- iPhone
- iPhone Pro Max
- iPad
- Mode portrait
- Mode paysage

L'objectif est d'éviter une interface conçue pour une seule taille fixe.

2. Éviter les dimensions fixes

À éviter lorsque ce n'est pas nécessaire :

```
Text("Bienvenue")
    .frame(width: 300, height: 50)
```

Une largeur fixe peut devenir problématique sur un petit écran.

On préfère souvent :

```
Text("Bienvenue")
    .frame(maxWidth: .infinity)
```

La vue utilise alors l'espace disponible.

3. Utiliser maxWidth

Exemple :

```
Button("Continuer") {
}
.frame(maxWidth: .infinity)
```

Avec :

```
.padding()
```

le bouton peut s'adapter à la largeur de l'écran.

4. Spacer

Spacer permet de répartir automatiquement l'espace disponible.

```
HStack {
    Text("Profil")
    Spacer()
    Image(systemName: "person.circle")
}
```

La distance entre les deux éléments s'adapte automatiquement.

5. Mise en page flexible

Exemple :

```
VStack(spacing: 20) {
    Text("Bienvenue")
        .font(.title)
    Text("Connectez-vous pour continuer")
        .multilineTextAlignment(.center)
    Button("Connexion") {
    }
    .buttonStyle(.borderedProminent)
}
.padding()
```

Cette interface fonctionne sur plusieurs tailles d'écran sans dimensions fixes.

6. Texte sur plusieurs lignes

Un texte peut automatiquement utiliser plusieurs lignes.

```
Text(
    "Cette application permet de gérer votre profil et vos préférences."
)
    .multilineTextAlignment(.center)
```

On peut aussi limiter le nombre de lignes :

```
.lineLimit(2)
```

7. minimumScaleFactor

Pour permettre à un texte de réduire légèrement sa taille si nécessaire :

```
Text("Informations importantes")
    .font(.title)
    .minimumScaleFactor(0.7)
```

Cela peut éviter que le texte soit coupé.

8. layoutPriority

Lorsque plusieurs vues se disputent l'espace disponible :

```
HStack {
    Text("Nom complet de l'utilisateur")
        .layoutPriority(1)
    Image(systemName: "person.circle")
}
```

La vue ayant une priorité plus élevée reçoit davantage d'espace.

9. Safe Area

Certaines zones de l'écran sont réservées au système :

- Encoche
- Dynamic Island
- Barre d'accueil

Ce sont des zones physiques ou système de l'écran d'un iPhone qu'une interface SwiftUI doit respecter.

- **Encoche (notch)** : zone noire située en haut de certains iPhones, qui contient notamment la caméra avant et les capteurs. Il ne faut pas placer du contenu important dessous.
- **Îlot dynamique (Dynamic Island)** : zone interactive en haut de certains modèles récents d'iPhone. Elle remplace l'encoche sur ces appareils et peut afficher des informations temporaires comme un appel, une minuterie ou une activité en cours.
- **Barre d'accueil (Home Indicator)** : petite barre horizontale située en bas des iPhones sans bouton Home. Elle sert notamment à revenir à l'écran d'accueil avec un geste vers le haut.

SwiftUI respecte généralement automatiquement ces zones.

Exemple :

```
VStack {
    Text("Contenu")
}
```

10. ignoresSafeArea

Dans certains cas, on souhaite que l'arrière-plan occupe tout l'écran :

```
ZStack {
    Color.blue
        .ignoresSafeArea()
    Text("Bienvenue")
}
```

Ici, seule la couleur ignore la Safe Area.

11. Attention à ignoresSafeArea

Pour le contenu principal, il est souvent préférable de conserver la Safe Area.

À éviter :

```
VStack {
    Text("Bouton important")
}
.ignoresSafeArea()
```

Le contenu pourrait se retrouver dans une zone difficile à utiliser.

12. GeometryReader

GeometryReader permet d'obtenir les dimensions disponibles.

```
GeometryReader { geometry in
    Text("Largeur : \(geometry.size.width)")
}
```

On peut utiliser :

```
geometry.size.width
geometry.size.height
```

pour adapter l'interface.

13. Exemple avec GeometryReader

```
GeometryReader { geometry in
    VStack {
        Text("Largeur disponible")
        Text("\(geometry.size.width)")
    }
    .frame(
        width: geometry.size.width
    )
}
```

14. Adapter selon la largeur

Exemple :

```
GeometryReader { geometry in
    if geometry.size.width > 600 {
        HStack {
            Image(systemName: "person.circle")
            Text("Profil")
        }
    } else {
        VStack {
            Image(systemName: "person.circle")
            Text("Profil")
        }
    }
}
```

Sur un écran large :

```
Image  Profil
```

Sur un écran étroit :

```
Image
Profil
```

15. ViewThatFits

SwiftUI peut essayer plusieurs présentations et choisir celle qui tient dans l'espace disponible.

```
ViewThatFits {
    HStack {
        Text("Accueil")
        Text("Profil")
        Text("Paramètres")
    }
    VStack {
        Text("Accueil")
        Text("Profil")
        Text("Paramètres")
    }
}
```

SwiftUI utilise la première disposition qui peut tenir dans l'espace disponible.

16. Exemple — Vue de profil adaptative

```swift
struct ProfilAdaptatifView: View {
    var body: some View {
        ViewThatFits {
            HStack(spacing: 20) {
                profilImage
                profilInfo
            }
            VStack(spacing: 20) {
                profilImage
                profilInfo
            }
        }
        .padding()
    }

    private var profilImage: some View {
        Image(systemName: "person.circle.fill")
            .font(.system(size: 80))
    }

    private var profilInfo: some View {
        VStack(alignment: .leading) {
            Text("Alex Martin")
                .font(.title2)
                .bold()
            Text("Développeur iOS")
                .foregroundStyle(.secondary)
        }
    }
}
```

17. Orientation portrait et paysage

SwiftUI recalcule automatiquement la mise en page lorsque l'espace disponible change.

Par exemple :

Orientation	Espace
Portrait	Moins de largeur
Paysage	Plus de largeur

On évite donc autant que possible de coder directement :

```
si portrait
si paysage
```

On préfère généralement adapter l'interface selon l'espace disponible.

18. Exemple de carte adaptative

```
struct ProduitView: View {
    var body: some View {
        VStack(alignment: .leading, spacing: 12) {
            Image(systemName: "laptopcomputer")
                .font(.system(size: 50))
            Text("MacBook")
                .font(.title2)
                .bold()
            Text(
                "Ordinateur portable pour le développement."
            )
                .foregroundStyle(.secondary)
            Button("Voir le produit") {
            }
            .buttonStyle(.borderedProminent)
            .frame(maxWidth: .infinity)
        }
        .padding()
        .frame(maxWidth: .infinity)
    }
}
```

L'interface ne dépend d'aucune largeur précise.

19. Bonnes pratiques

Préférer :

```
.frame(maxWidth: .infinity)
.padding()
Spacer()
ViewThatFits
```

plutôt que :

```
.frame(width: 375)
```

Éviter de supposer qu'un appareil possède toujours la même taille.

20. À retenir

Une interface adaptative doit :

- Utiliser l'espace disponible
- Éviter les dimensions fixes inutiles
- Supporter plusieurs tailles d'écran
- Respecter la Safe Area

- S'adapter aux changements de largeur

Outils importants :

```
Spacer
frame(maxWidth:)
GeometryReader
ViewThatFits
layoutPriority
minimumScaleFactor
```

21. Exercice — Profil adaptatif

Créer une vue :

```
ProfilAdaptatifView
```

contenant :

- Une image
- Un nom
- Un courriel
- Un bouton

Sur un écran suffisamment large :

```
Image  Informations
```

Sur un écran plus étroit :

```
Image
Informations
```

Utiliser :

```
ViewThatFits
```

pour choisir automatiquement entre HStack et VStack.

22. Exercice — Arrière-plan adaptatif

Ajouter un arrière-plan qui occupe tout l'écran avec :

```
ignoresSafeArea()
```

tout en gardant le contenu à l'intérieur de la Safe Area.